GUIA VISUAL 050

# CRIPTOGRAFIA DE DISCO LUKS E A PERSISTÊNCIA

*O notebook fica no táxi. Com um live USB, ler tudo leva cinco minutos:* ***permissão é conferida pelo kernel que está rodando*** *— outro kernel, outro root.*

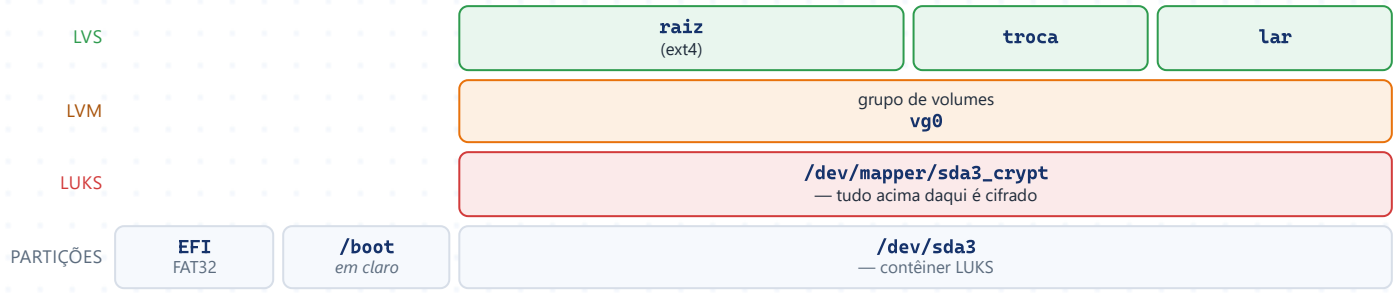
050
GUIAS LINUX

## DUAS CHAVES — POR QUE TROCAR A SENHA É INSTANTÂNEO




**Dados do volume.** Cada senha só decifra uma **cópia** da mestra num *slot* — daí trocar a senha reescrever poucos KB em vez de recifrar o disco, e várias pessoas terem senhas distintas para o mesmo volume. LUKS1 tem 8 *slots*; **LUKS2, 32**, com Argon2id (caro em tempo **e** em memória). E daí também: **apagar o cabeçalho destrói todas as cópias de uma vez**.

## A PILHA DE UMA INSTALAÇÃO CIFRADA — RAID → LUKS → LVM




**LUKS embaixo, LVM em cima:** uma senha abre um contêiner e protege todos os volumes lógicos — a ordem inversa pediria uma senha por volume, sem ganho. A **EFI e a `/boot` ficam fora por necessidade**: alguém tem de carregar o kernel e o *initramfs* que vão pedir a senha. É a superfície que sobra — com acesso físico repetido dá para alterar o *initramfs* e capturar a senha (a "camareira maligna"). Defesa: *Secure Boot* com chaves próprias ou selar a chave no TPM (`systemd-cryptenroll --tpm2-device=auto`).

**Perder o cabeçalho é perder os dados — não existe recuperação.** Ele ocupa poucos MB no início do volume e guarda **todas** as cópias da chave mestra: um `dd` nos primeiros blocos, um `wipefs -a` ou uma tabela de partição reescrita com outro deslocamento e nenhuma senha do mundo abre o volume. Faça `luksHeaderBackup`, guarde **fora do disco protegido**, trate como material sensível — e **teste a restauração** antes de precisar dela; note que um cabeçalho antigo **revive senhas já removidas**. O `luksAddKey` exige uma senha válida: **removeu o último *slot*, acabou**. E o `reencrypt`, que troca a chave mestra com o sistema no ar, é a operação mais longa e arriscada — backup completo antes, sem exceção.

## RESUMO RÁPIDO

- ✓ **Permissão não protege contra acesso físico.** Ela é conferida pelo kernel em execução contra o usuário autenticado — outro sistema, iniciado como `root`, ignora tudo.
- ✓ **Opere sempre em `/dev/mapper/<nome>`**, nunca no dispositivo cru: formatar o original destrói o cabeçalho.
- ✓ **Dois UUIDs:** o do contêiner vai no `crypttab`, o do sistema de arquivos vai no `fstab`. Trocá-los trava o boot esperando um dispositivo que nunca aparece.
- ✓ **Com AES-NI a criptografia não é o gargalo** — o `cryptsetup benchmark` entrega mais do que um SSD SATA sustenta. Em ARM sem aceleração, compare com `xchacha20`.
- ✓ `discard` em volume cifrado revela **quais blocos estão em**

## NO KALI

**Persistência cifrada é o padrão para pendrive.** Crie a 3ª partição depois das duas da imagem, `luksFormat`, formate em **ext4** com rótulo `persistence` e grave `/ union` no `persistence.conf`. O *initramfs* acha a partição **pelo rótulo** — imune à troca de letras a cada máquina — e exFAT/FAT32 não servem: não representam dono nem permissão, e o sistema não sobe. Num equipamento de trabalho, criptografia **não é opcional**: ele carrega capturas, credenciais de teste e dados de cliente. O `cryptsetup` do Kali traz as **chaves *nuke*** (`luksAddNuke`): digitada no boot, a senha apaga todos os *slots* em vez de abrir — protege confidencialidade contra acesso não autorizado, mas destruir dados para obstruir investigação legalmente autorizada é crime, e sem backup do cabeçalho é definitivo também para o dono. Em perícia, a chave mestra vive